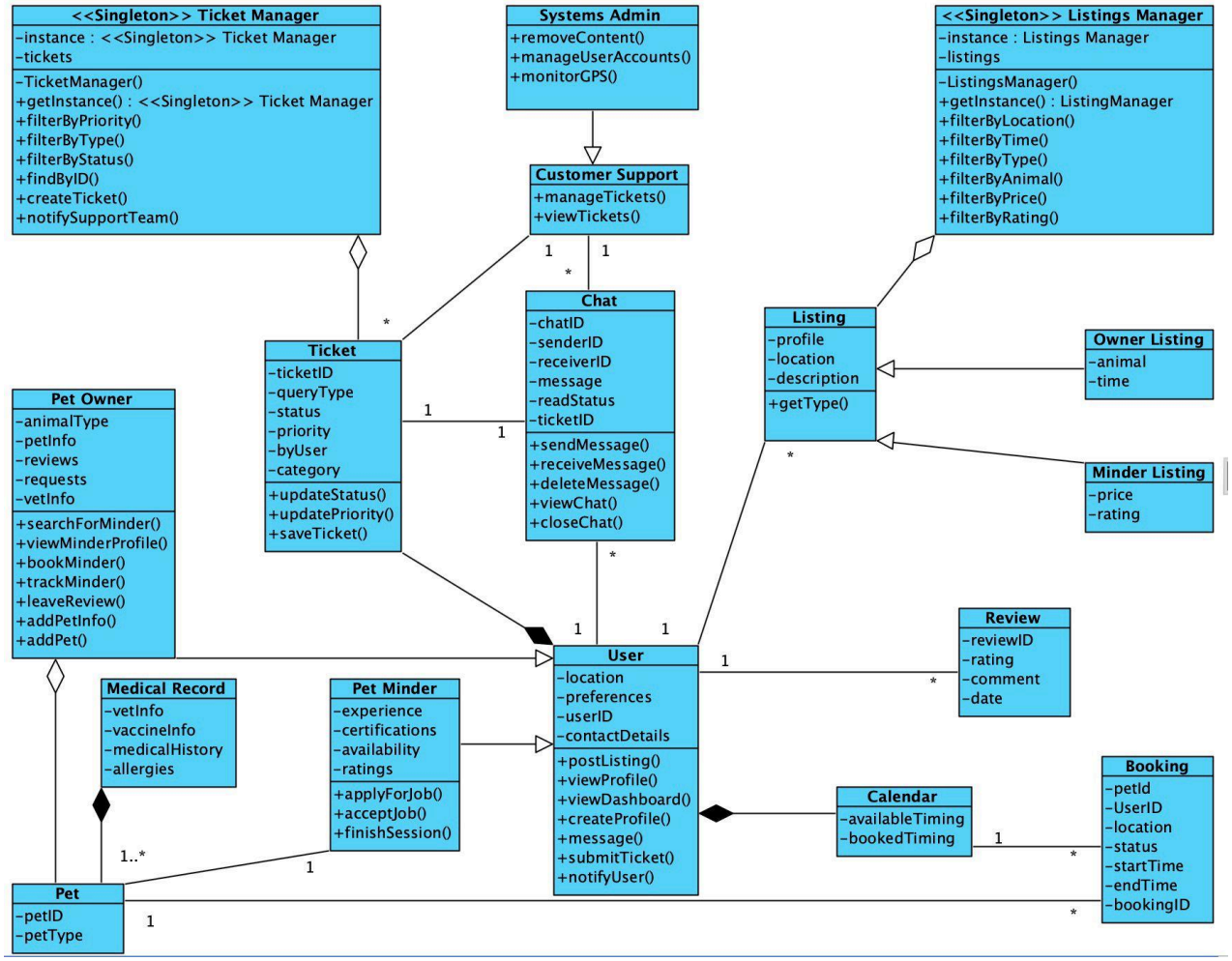
Group 26

Pet Minder App

ECS506U Software Engineering Group Project

Design Report

1. Class Diagram



2. Traceability matrix

The traceability matrix maps each functional requirement identified during requirements elicitation to the corresponding classes in the system's UML class diagram. This ensures that all requirements are supported by the system design.

Class	**Requirement (ID → short requirement)**	**Brief explanation (how design fulfils it)**
User	RQ1, RQ25, RQ26, RQ28, RQ29, RQ32, RQ45, RQ46	User stores shared account data including location, contact details, login credentials, communication preferences and account lifecycle operations.
PetOwner	RQ8, RQ18, RQ21, RQ24, RQ30, RQ48, RQ49	PetOwner represents users who own pets and request minding services, including booking requests, cancellations, viewing bookings and managing favourites.
PetMinder	RQ3, RQ9, RQ10, RQ11, RQ12, RQ20, RQ23, RQ27, RQ51	PetMinder extends User and stores service-specific attributes such as supported animal types, certifications, availability and pricing.
Pet	RQ4, RQ15, RQ30	Pet stores pet profile data including petID, type and breed and allows owners to manage multiple pets.
MedicalRecord	RQ5	MedicalRecord stores veterinary information such as vaccine details, vet information, medical history

		and allergies linked to a Pet.
Listing / OwnerListing / MinderListing	RQ33, RQ34, RQ35, RQ36	Listing represents job or service postings in the system, with specialised subclasses for owner listings and minder service listings.
ListingsManager	RQ6, RQ7, RQ16, RQ17	ListingsManager manages retrieval and filtering of listings and profiles based on search parameters such as location, rating and animal type.
Booking	RQ8, RQ18, RQ21, RQ24, RQ52	Booking represents scheduled pet-minding sessions including time, location, status and associated users.
Calendar	RQ12, RQ20	Calendar manages available and booked time slots for Pet Minders to prevent scheduling conflicts.
Chat	RQ13	Chat stores communication messages exchanged between users regarding bookings or other queries.
Review	RQ31	Review stores rating and feedback information after completion of a pet-minding session.
Ticket	RQ37, RQ42	Ticket stores user queries or complaints including ticket ID, priority, status and description.

TicketManager	RQ39	TicketManager provides operations to manage and sort support tickets.
CustomerSupport	RQ40	CustomerSupport actors manage and view user tickets that are not categorised as technical issues.
SystemsAdmin	RQ38, RQ41, RQ43, RQ44, RQ47	SystemsAdmin manages administrative tasks such as moderating users, resolving tickets and monitoring system operations.
Payment	RQ50	Payment handles secure in-app transaction processing between Pet Owners and Pet Minders.

3. Design Discussion

Vet Entity
“Vets” will not be a class; instead, their information will be an attribute of the Pet’s Medical Record. This is because, as a third-party entity, a Vet's information is associated with a Pet, and Vets cannot act as a User in our system. If a Pet Owner or Minder needs to contact one, they would just look at the Pet’s information to call them. Therefore, Vet Information will be basic information attached to their medical record and not its own entity.

Payment system
The system will not have a payment class, as we have designed it to be arranged privately between the Pet Owner and Pet Minder. This design choice allows users to message each other about the job, and if both parties agree, they will send the bank details for an online bank transfer. This allows users to make payments and simplifies our design. As we are a prototype, most prebuilt payment functions cost money to implement, which is not feasible at the moment without crowdfunding or investment. We would also not need to think about regulatory requirements and financial compliance in both app stores. This system allows us to focus on the core mission of Safepaws, connecting Pet Owners to Pet Minders. This works as one of our requirements is to keep information secure and have an encrypted chat, allowing users to communicate securely.

Aggregation:
Our class diagram highlights multiple aggregations. One example of this is the aggregation between Pet and Pet Owner. Pet Owners act as a parent entity while Pets are the child object. A Pet Owner can own pets, and pets can exist even if their owner's object is removed from the system, which means it is not a composition. For example, a Pet may later be adopted by another Owner, or a Pet Owner deletes their SafePaw's account while the Pet information stays in our database for data purposes.

This aligns with our Domain Analysis and Requirement Elicitation reports, as we have consistently treated a Pet as its own entity within our system. The owner can register a separate pet profile that contains their own data, such as medical records. Beyond this, pets are involved in bookings, GPS tracking and other relationships, which means it makes sense that it would be a distinct class rather than a field of the Pet Owner.

Association:
We described the Booking and Pet relationship as an association, with multiplicity of 1 to *. As booking and pet are separate classes that are not directly tied to each other, this relationship is modelled as an association. A single pet can have multiple bookings on different dates, but each booking will always refer to one pet. As stated in the Requirements Elicitation, Pet Owners can add bookings to multiple pets, and bookings do not have a parent-child relationship with Pets.

Many of our associations have different multiplicities like Pet Owners and Pets having a multiplicity of 1 to 0..*. This is because Pet Owners can create multiple pet profiles. However, it is possible that a Pet Owner could temporarily have no pets. Like a Pet passing away or being adopted by another owner. Therefore, allowing the multiplicity to be 1 to 0..*. This also aligns with our Requirements Elicitation report, specifically Requirement 30.

Composition:
An example of composition we used was between Pet and MedicalRecord. We needed to use this type of relationship, as without the Pet, the medical record would not exist. Every Pet should contain individual medical information which includes vaccination history, allergies and vet information.

As described in our Domain Analysis, medical information of pets should be stored within their profiles. This means that without a pet profile, it would not be able to exist independently.

Generalisation:
We used generalisation for many classes, one example being generalisation of User and Pet Owner. Pet Owner inherits all the capabilities of User while also having its distinctive behaviours and attributes. The Pet Owner class extends its functionalities like being able to addPetInfo() or bookMinder(). This allows users with similar basic functionality to have the same superclass, and inherit those attributes whilst having their own unique ones, reducing duplicate operations.

As described in the Requirement Elicitation report, different users have different roles which support generalisation. Some requirements highlight how every user can register an account and create their own profile, while Pet Owners specifically can manage pets and schedule bookings.

There are a few differences between our Design and Domain class diagram, purely since we have now extended the scope of the project to create a fully functioning Pet Minding application. Besides the changes below, we have just added a lot more attributes to each class which aligns with our Requirements Elicitations report and feedback from previous reports. So that every key entity could carry out their needs.

Medical records were specified before in the Domain Model, but have been refined to have specific attributes, like allergies. We have created a review class, so that both users are able to comment on other People's jobs, building reputation and ratings and also potentially flagging suspicious or bannable behaviour with which Admin could investigate. We introduced a ticket system, which allows our System Admin to handle customer support and technical problems, maximising user experience and business operations. We realised that we do not mind having pets outside of dogs and cats, so we have decided that its PetType would just be an attribute to Pet, instead of previously generalising into a Dog and Cat class.

We added a singleton pattern for both Listing Manager and Ticket Manager. This ensures only one instance of the classes exist in the system. As these managers have access to global data that needs to be managed consistently across the system, it would be efficient to implement them as singletons, as this will remove duplicate and contradictory data. If multiple ticket managers existed, different parts of the system would have different ticket statuses, which can create complexity. This allows both Tickets and Listings to be tracked and updated consistently. As we use the global access method getInstance(), we don't need to create new objects each time. Overall, this design pattern allows for data consistency, resource management and better system control.

4. Sequence Diagrams

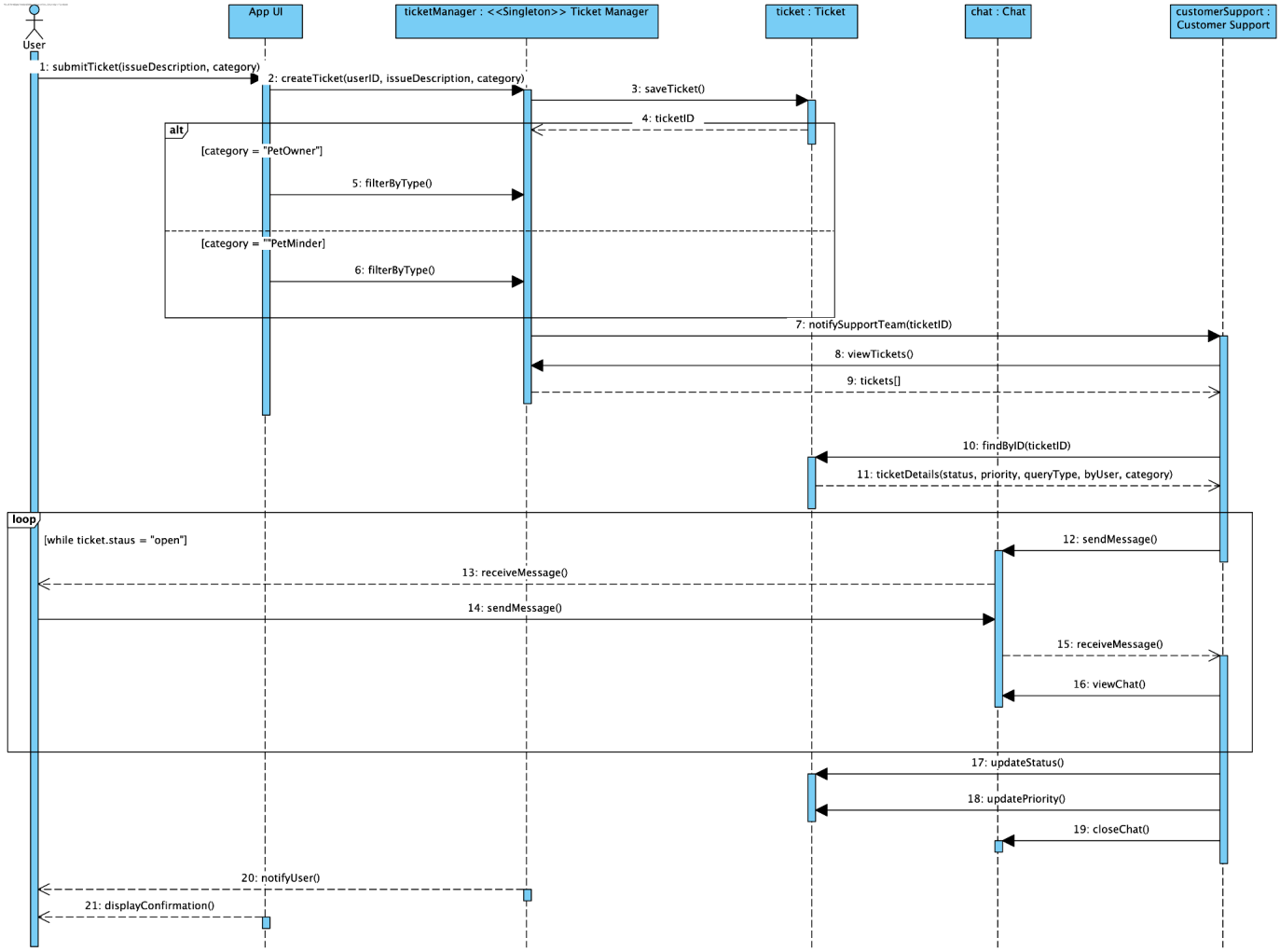
Use case - Customer Support Ticketing Service

Explanation and Prerequisites:

The sequence diagram outlines how a User which is either a Pet Minder or Pet Owner submits a support ticket and a Customer Support agent resolves it through the in app ticketing service. Once the User submits a ticket with a description and category, the TicketManager creates and saves the ticket and routes it to the correct queue based on the user type. The Customer Support agent is notified which allows them to view the ticket queue and open the ticket. The agent and user are then able to communicate back and forth until the ticket is resolved. Once resolved the agent updates the ticket status and priority. The chat is then closed and a notification confirmation is sent to the User.

The diagram assumes that:

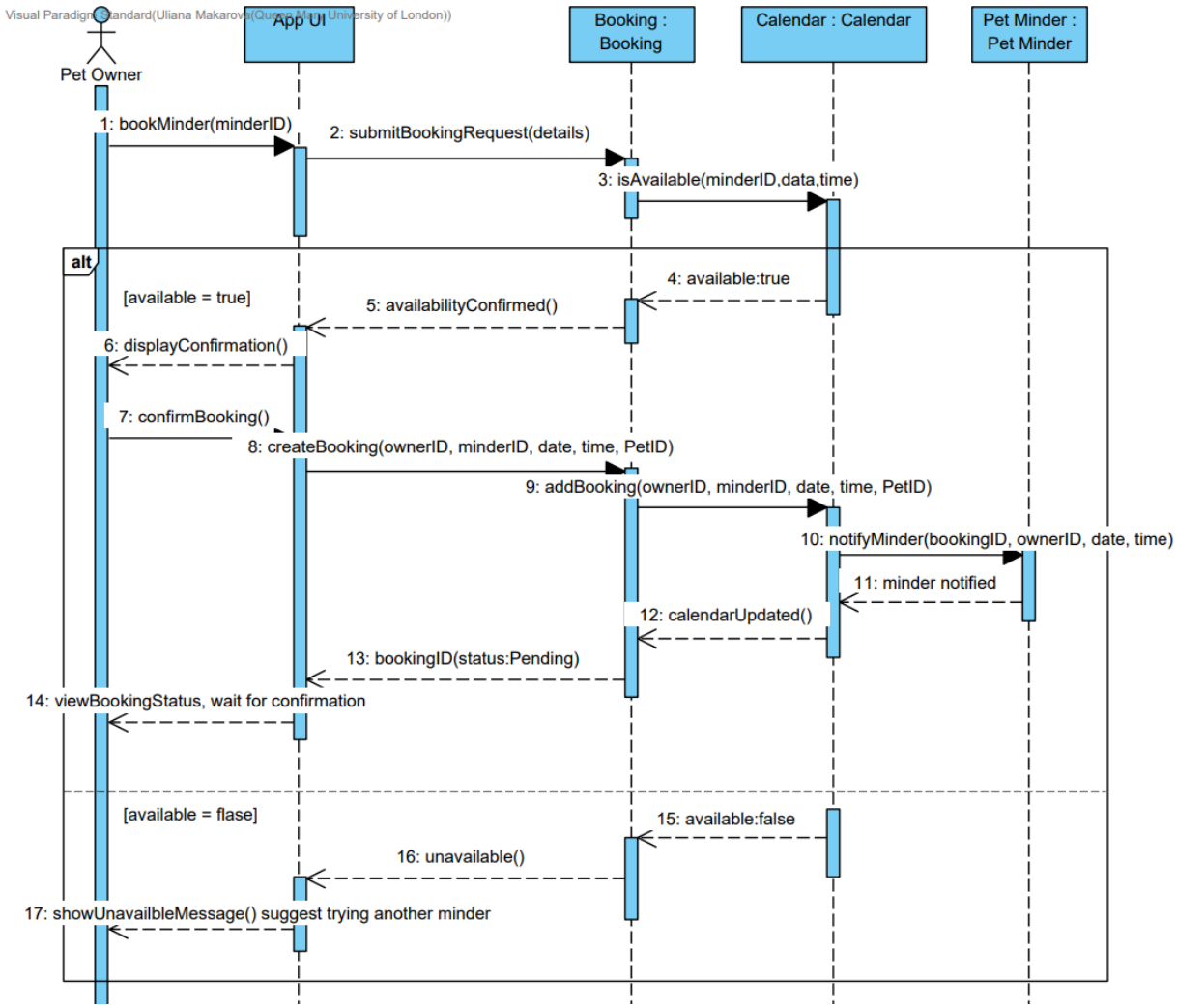
- The User is logged in with a valid account
- The User is experiencing a technical or service related issue they wish to report
- The TicketManager instance is active and available to handle relevant tickets
- A Customer Support agent is available and assigned to handle tickets
- The Chat session is open for the duration of the ticket and is closed only once the ticket is resolved


User
App UI
ticketManager : <<Singleton>> Ticket Manager
ticket : Ticket
chat : Chat
customerSupport : Customer Support
1: submitTicket(issueDescription, category)
2: createTicket(userID, issueDescription, category)
3: saveTicket()
4: ticketID
alt
[category = "PetOwner"]
5: filterByType()
[category = ""PetMinder]
6: filterByType()
7: notifySupportTeam(ticketID)
8: viewTickets()
9: tickets[]
10: findByID(ticketID)
11: ticketDetails(status, priority, queryType, byUser, category)
loop
[while ticket.staus = "open"]
12: sendMessage()
13: receiveMessage()
14: sendMessage()
15: receiveMessage()
16: viewChat()
17: updateStatus()
18: updatePriority()
19: closeChat()
20: notifyUser()
21: displayConfirmation()

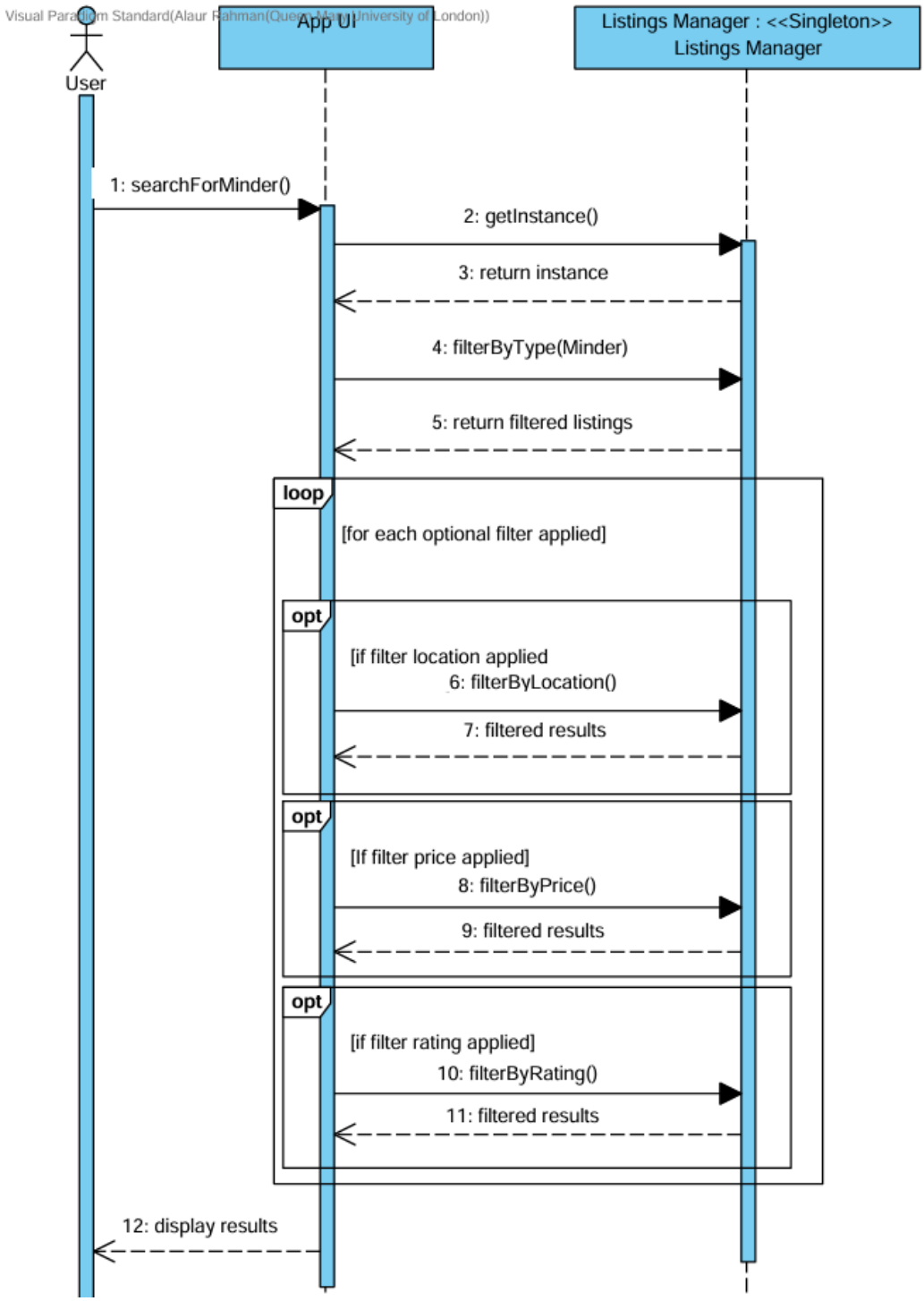
Use case - Request booking

A Pet Owner can request a booking with a selected Pet Minder for a specific date and time.

Preconditions: The user is logged in and the system is online. The Pet Owner has used the Search for Minder interface and is shown the final results of Pet Minders. The Pet Owner has chosen one of the Pet Minders profile to request a booking.



Use Case - Search For Minder

This use case depicts a Pet Owner looking for a Pet Minder using the search interface.

Preconditions: The Pet Owner must have registered and logged in to their account. The system must also be online and functional.